Machine Learning Assignment (M2026)

Classification, Regression, K-Means Clustering and Principal Component Analysis

Indian Institute of Information Technology, Sri City
Course: Machine Learning
Name: Gokul
Programme: B.Tech Computer Science and Engineering
Date of submission: 31 October 2026

Summary

This report answers the three assignment questions on standard UCI datasets. Part 1 compares K-nearest neighbours (with cross-validation), a decision tree and an RBF-kernel support vector machine on a ten-class product-title classification task. Part 2 compares gradient-descent linear regression, polynomial regression, ridge and lasso regression on hourly traffic volume. Part 3 implements K-means clustering and principal component analysis from scratch on gas-turbine sensor data. For each part the report states the data, the method, the results, and the points where the results should be read with care. The optional CNN bonus was not attempted.

Table 1. Summary of the main results.

Part	Dataset	Main result
1. Classification	PriceRunner, 35,311 titles, 10 classes	SVM has the highest accuracy (95.30%); KNN has the highest macro ROC-AUC (0.9933) and is the best classifier by ROC analysis
2. Regression	Metro Interstate Traffic Volume, 48,204 hourly records	Linear model test R^2 about 0.70 (GD: 0.6975); ridge $\alpha = 100$ and lasso $\alpha = 10$ chosen by time-series cross-validation
3A. K-means	Gas Turbine emissions, 36,726 records	WCSS computed for K = 2 to 6; elbow is gradual, K = 3 used for interpretation
3B. PCA	Gas Turbine emissions, 11 features	2 components keep 67.90% and 3 components keep 77.35% of the variance

Contents

1. Overview 2
2. Part 1: Multiclass Classification 2
3. Part 2: Regression 7
4. Part 3A: K-Means Clustering from Scratch 11
5. Part 3B: Principal Component Analysis from Scratch 13
6. Limitations 15
7. Reproducibility and Files 16
8. References 16

1. Overview

1.1 Datasets

All three datasets come from the UCI Machine Learning Repository. They are standard public datasets, so no dataset change was needed for any part.

Table 2. Datasets used in the assignment.

Part	Dataset (UCI ID)	Records	Task
1	PriceRunner Product Classification (837)	35,311	Predict one of 10 product categories from the product title (text)
2	Metro Interstate Traffic Volume (492)	48,204	Predict hourly traffic volume (vehicles per hour) from weather and calendar information
3	Gas Turbine CO and NOx Emissions (551)	36,733 raw, 36,726 used	Unsupervised analysis of 11 continuous hourly sensor variables, 2011 to 2015

1.2 Setup

The code is written in Python with NumPy, pandas, scikit-learn and matplotlib. A fixed random seed (42) is used everywhere. K-means, principal component analysis and gradient-descent linear regression are implemented from scratch in NumPy; KNN, the decision tree, the SVM, polynomial regression, ridge and lasso use scikit-learn. In every part the test data are used only for the final evaluation: preprocessing statistics, vocabularies and hyperparameters are all fitted on training data only. Every number in this report is produced by the scripts in the code submission (Section 7).

2. Part 1: Multiclass Classification

2.1 Data and preprocessing

The PriceRunner data contain 35,311 product offers. The input is the product title and the target is one of ten categories. The classes are not perfectly balanced: the largest class (Fridge Freezers) has 5,501 records and the smallest (Freezers) has 2,212, a ratio of about 2.5 to 1. Because of this, macro-averaged metrics are reported, which give every class the same weight.

Table 3. Class distribution of the PriceRunner dataset.

Category	Records	Share (%)
Fridge Freezers	5,501	15.6
Mobile Phones	4,081	11.6
Washing Machines	4,044	11.5
CPUs	3,862	10.9
Fridges	3,584	10.1
TVs	3,564	10.1
Dishwashers	3,424	9.7
Digital Cameras	2,697	7.6
Microwaves	2,342	6.6
Freezers	2,212	6.3

The data are split 80% / 20% into a training set of 28,248 titles and a test set of 7,063 titles using stratified sampling, so each class keeps the same proportion in both sets. Titles are converted to TF-IDF vectors with sublinear term frequency, unigrams and bigrams, a minimum document frequency of 2 and at most 5,000

features. The vectorizer is fitted on the training titles only and then applied to the test titles:

tf-idf(t, d) = (1 + ln tf(t, d)) × (ln((1 + N) / (1 + df(t))) + 1), followed by L2 normalisation of each title vector.

2.2 Classifiers and cross-validation

K-nearest neighbours. Cosine distance with brute-force search is used, which suits sparse TF-IDF vectors. The number of neighbours k in {3, 5, 7, 11, 15} and the voting rule (uniform or distance-weighted) were selected by 5-fold stratified cross-validation on the training set. The best setting is k = 15 with distance weighting (CV accuracy 93.79%).

Table 4. KNN 5-fold cross-validation accuracy (%) on the training set.

k	Uniform voting	Distance-weighted voting
3	92.79	93.38
5	92.73	93.53
7	92.73	93.63
11	92.67	93.71
15	92.69	93.79

Decision tree. The grid was split criterion {Gini, entropy} × maximum depth {10, 20, 30, unlimited} × minimum samples to split {2, 5, 10}, again with 5-fold stratified cross-validation. The best setting is Gini, unlimited depth and minimum split size 2 (CV accuracy 93.98%). The fitted tree has depth 241 and 874 leaves.

Support vector machine. An RBF kernel was used with C in {0.1, 1, 10} and γ in {scale, 0.01}. Because the kernel SVM is expensive on 28,248 titles, the grid search used a stratified random subset of 10,000 training titles with 3-fold cross-validation. The selected setting (C = 10, γ = scale) was then trained on the full training set. To obtain class probabilities for the ROC analysis, the final SVM is wrapped in a 3-fold sigmoid (Platt) calibration.

Table 5. SVM 3-fold cross-validation accuracy on the 10,000-title subset.

C	γ	CV accuracy (%)	Std (points)
10	scale	93.98	0.25
1	scale	93.84	0.16
10	0.01	92.84	0.15
0.1	scale	85.91	0.16
1	0.01	70.56	0.20
0.1	0.01	15.58	0.02

2.3 Test-set results

Table 6 compares the three classifiers on the 7,063 held-out test titles. Precision, recall and F1-score are macro-averaged over the ten classes and ROC-AUC is the macro-average one-vs-rest area under the curve computed from predicted class probabilities.

Table 6. Classification metrics on the test set (macro averages).

Classifier	Accuracy (%)	Precision	Recall	F1-score	ROC-AUC
KNN (k = 15, cosine, distance-weighted)	93.86	0.9425	0.9356	0.9385	0.9933
Decision tree (Gini, unlimited depth)	94.14	0.9426	0.9408	0.9414	0.9859
SVM (RBF, C = 10, γ = scale)	95.30	0.9551	0.9534	0.9540	0.9908

The SVM has the best accuracy, precision, recall and F1-score. The decision tree and KNN are about 1.2 and 1.4 percentage points lower in accuracy. Support-weighted averages differ from the macro averages by at most 0.0034, so the class imbalance does not change the ranking.

2.4 Confusion matrices

Figures 1 to 3 show the confusion matrices of the three classifiers on the test set (rows are true classes, columns are predicted classes). All three models separate CPUs, digital cameras, mobile phones and TVs almost perfectly; for example the SVM classifies 771 of 773 CPUs correctly. Almost all remaining errors involve the three refrigeration categories and dishwashers or washing machines. For the SVM the largest errors are 42 fridge freezers predicted as fridges, 32 fridges predicted as fridge freezers, 23 fridge freezers predicted as dishwashers and 20 freezers predicted as fridges. The decision tree and KNN show the same pattern (KNN: 52 fridges predicted as fridge freezers and 44 the other way round). A likely reason is that titles of these categories share the same brand, capacity and colour words and differ in only one or two category words; this explanation was not tested separately.

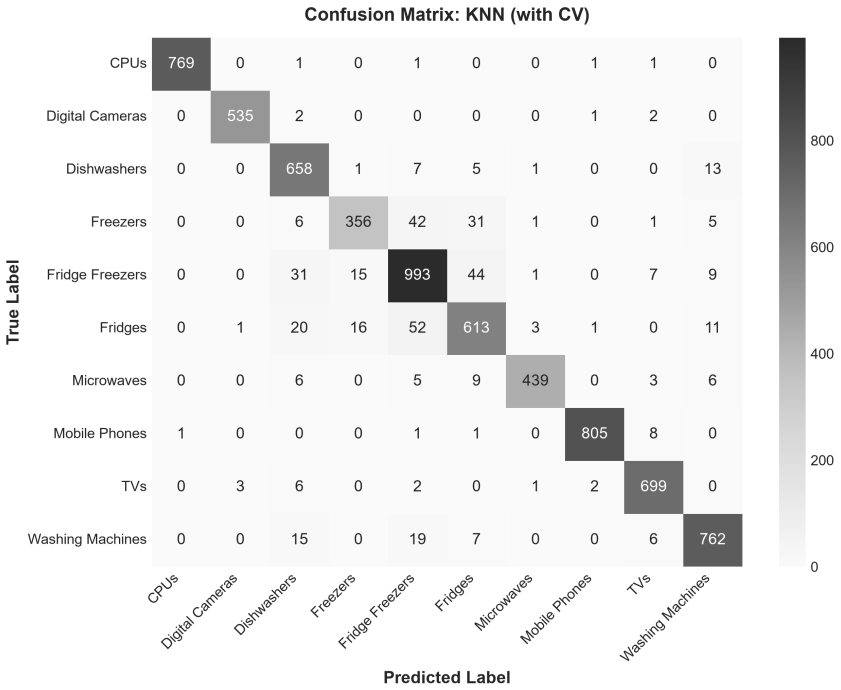


Figure 1. Confusion matrix of the KNN classifier on the test set.

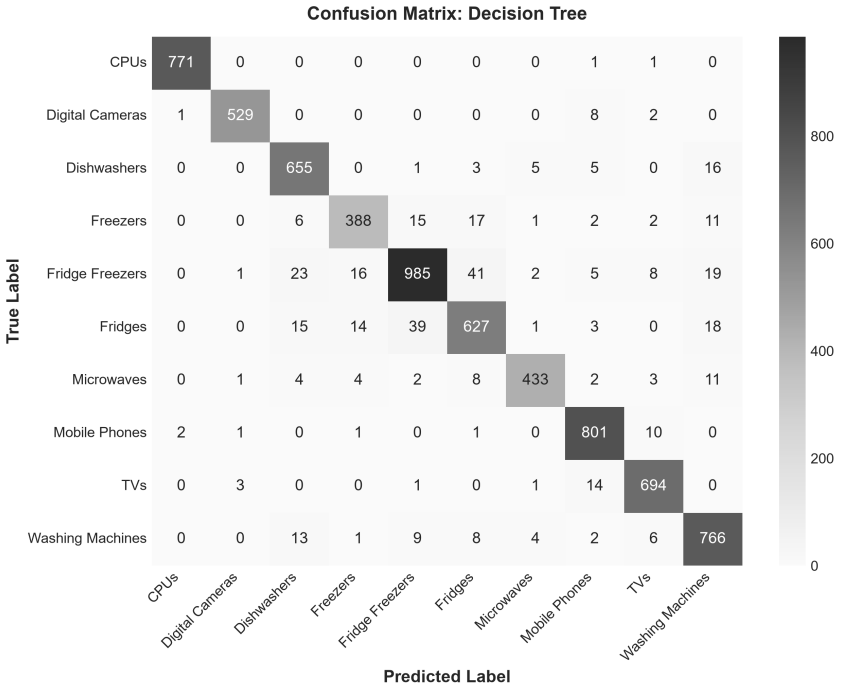


Figure 2. Confusion matrix of the decision tree on the test set.

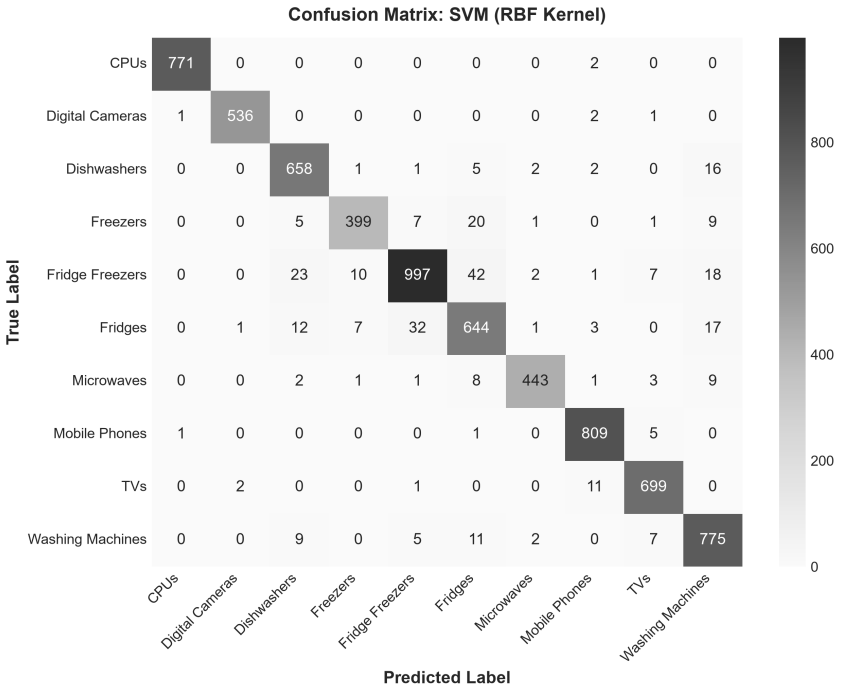


Figure 3. Confusion matrix of the RBF-kernel SVM on the test set.

2.5 ROC analysis and best classifier

For each class a one-vs-rest ROC curve is computed from the predicted probabilities, and the curves are averaged over the ten classes. Figure 4 shows the three macro-average curves. All three lie close to the top-left corner, so the differences are small.

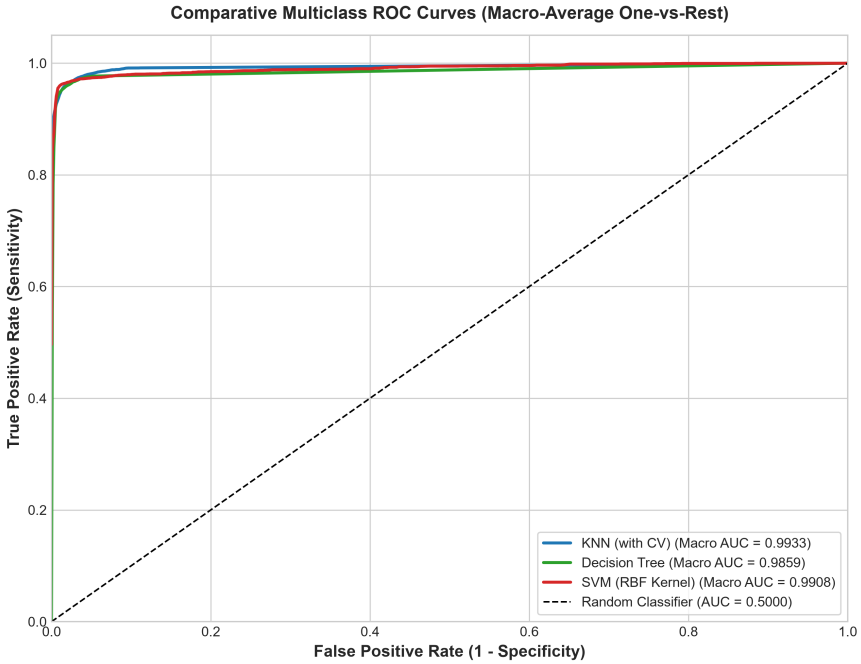


Figure 4. Macro-average one-vs-rest ROC curves of the three classifiers.

The macro ROC-AUC is 0.9933 for KNN, 0.9908 for the SVM and 0.9859 for the decision tree. Based on ROC analysis, as the assignment requires, KNN is therefore the best classifier. The margin over the SVM is small (0.0025), and the SVM is better on every threshold-based metric in Table 6, so the two models are close and the choice of criterion matters. Accuracy only uses the top-ranked class, whereas ROC-AUC measures how well the full probability ranking separates each class from the rest. A plausible reason for the decision tree's lower AUC is that an unpruned tree gives almost pure leaves and hence probabilities close to 0 or 1, which makes its ranking coarse; this was not tested separately.

2.6 Overfitting and model complexity

The three models control complexity in different ways: tree depth, the number of neighbours k, and the SVM parameters C and γ. Table 7 follows the decision tree as its depth limit is relaxed. Training accuracy rises steadily with depth. Test accuracy also rises, so for this dataset the extra capacity is useful and the unlimited tree is also the best setting chosen by cross-validation. The gap between training and test accuracy is about half a point for depth limits of 10 to 20, 1.1 points at depth 30 and 2.62 points for the unlimited tree. That gap is the signature of overfitting: the tree has memorised title patterns of the training set, but the effect is moderate because product titles contain many highly informative words.

Table 7. Decision tree complexity: effect of the depth limit.

Depth limit	Tree depth	Leaves	Train accuracy (%)	Test accuracy (%)	Gap (points)
10	10	32	74.33	73.79	0.54
15 (min split 10)	15	62	80.52	80.02	0.50
20	20	104	83.80	83.31	0.49
30	30	178	88.01	86.93	1.08
Unlimited (selected)	241	874	96.76	94.14	2.62

For KNN, small k gives a flexible, high-variance classifier and larger k smooths the decision boundary. In Table 4 the cross-validation accuracy of distance-weighted KNN increases monotonically from k = 3 to k = 15, and distance weighting beats uniform voting at every k, by 0.6 to 1.1 percentage points. Since k = 15 is the largest value in the grid, a still larger k might be slightly better; this was not explored. For the SVM (Table 5), C controls the margin penalty. A small C = 0.1 underfits: with γ = 0.01 the CV accuracy is 15.58%, which equals

the share of the largest class (Fridge Freezers, 15.6%), meaning the model predicts essentially one class. Increasing C to 10 improves accuracy to about 94%. C = 1 is only 0.14 points lower than C = 10, which is within the fold-to-fold spread, so the SVM is not very sensitive once C is large enough, and the regularised margin keeps the model from overfitting the high-dimensional text features.

2.7 Summary of Part 1

All required elements are covered: 80/20 split, KNN with cross-validation, decision tree and RBF SVM, confusion matrices, accuracy, precision, recall, F1-score and ROC-AUC, an overfitting and complexity discussion, and a best classifier by ROC analysis (KNN). The SVM is the most accurate model and KNN the best-ranked by ROC-AUC.

3. Part 2: Regression

3.1 Data and preprocessing

The dataset contains 48,204 hourly records of traffic volume on Interstate 94 together with weather and holiday information. Traffic is a time series, so a random split would put records from the same hour or day in both sets. The data are therefore sorted by time and split chronologically: the first 80% (38,563 records, October 2012 to 1 November 2017) form the training set and the last 20% (9,641 records, 1 November 2017 to 30 September 2018) form the test set.

Ten training records have a temperature of 0 K, which is a sensor error, and were replaced by the training median for the same month. One record has a rainfall of 9,831.3 mm in an hour, which is physically implausible (the next largest value is 55.63 mm); rainfall is capped at the 99.99th percentile of the training data (29.14 mm). Section 3.7 shows the effect of this choice. The model uses 23 features: the four continuous variables temperature, rainfall, snowfall and cloud cover (standardised with training statistics); sine and cosine encodings of hour of day, day of week and month; weekend and holiday flags; and 11 one-hot indicators of the weather condition.

3.2 Linear regression with gradient descent

The model is $\hat{y} = Xw + b$, trained by batch gradient descent on the cost $J(w, b) = (1/2N)\ \Sigma\ (\hat{y}_i - y_i)^2$ with the updates

$$w \leftarrow w - \eta\ (1/N)\ X^T(\hat{y} - y),\ b \leftarrow b - \eta\ (1/N)\ \Sigma(\hat{y}_i - y_i),$$

with learning rate $\eta = 0.05$, weights initialised with small Gaussian values and the bias at the mean of the training target. Figure 5 shows the cost over 1,000 iterations. The cost decreases smoothly, but the run stopped because the iteration limit was reached, not because the tolerance of 10^{-6} was met (the last change in cost was 0.71). The solution is therefore close to, but not exactly at, the minimum.

As a check, the result was compared with the closed-form ordinary least squares (OLS) solution on the same features. The test MSE of gradient descent is 1,171,125 against 1,173,326 for OLS, and the training MSE is 1,183,099 against 1,182,480. The predictions differ by 15.8 vehicles per hour on average (0.49% of the mean test traffic). OLS has the lower training error, as it must, while gradient descent is slightly better on the test set. A plausible reason is that stopping early acts like mild shrinkage (the weight norm is 2434 for gradient descent and 2535 for OLS); the difference of 0.19% is small and should not be over-interpreted.

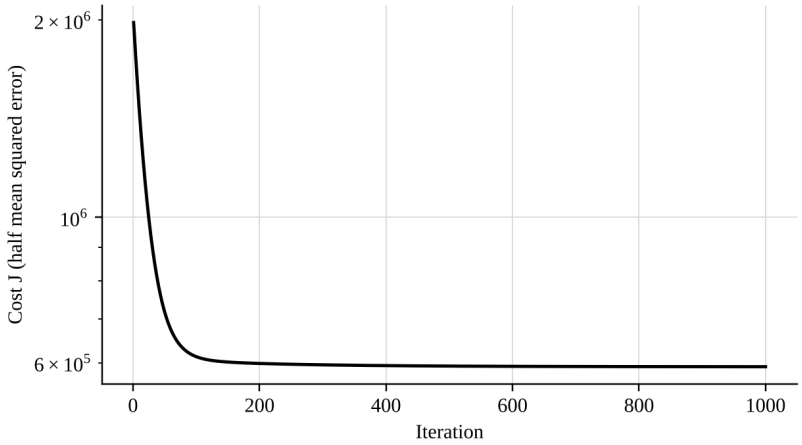


Figure 5. Gradient descent cost (log scale) against iteration.

3.3 Polynomial regression

Polynomial terms of degree 1 to 5 were generated for the four continuous features only (the cyclic, flag and weather columns are kept unchanged), and the model was fitted by least squares. This gives 23, 33, 53, 88 and 144 features. Degree 1 is the ordinary linear model. Table 8 gives the results on the full training set; the condition number is that of the design matrix without an intercept column.

Table 8. Polynomial regression on the full training set (N = 38,563).

Degree	Features	Condition number	Train MSE	Test MSE	Test RMSE	Test R^2
1	23	122	1,182,480	1,173,326	1,083.2	0.6969
2	33	4,509	1,178,655	1,166,996	1,080.3	0.6986
3	53	1.0×10^{20}	1,178,157	1,166,866	1,080.2	0.6986
4	88	9.1×10^{23}	1,175,760	1,164,935	1,079.3	0.6991
5	144	1.8×10^{27}	3,930,451	3,833,350	1,957.9	0.0099

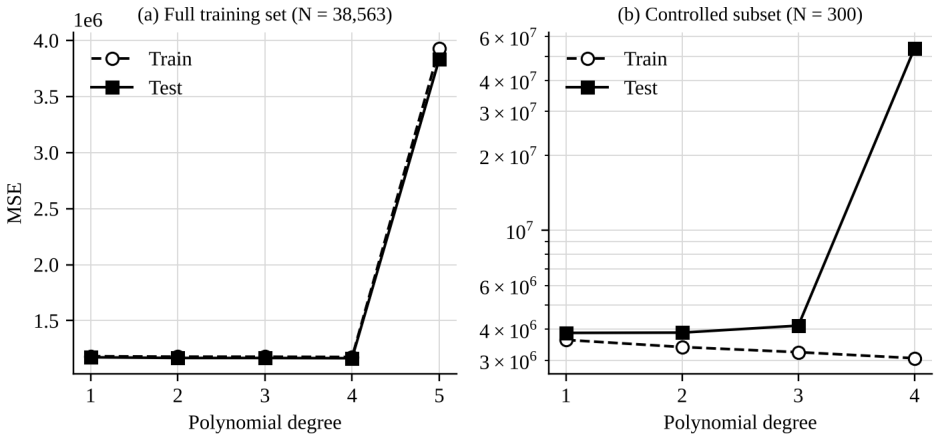


Figure 6. Training and test MSE against polynomial degree: (a) full training set, (b) controlled 300-record experiment of Section 3.4.

3.4 Underfitting and overfitting

Underfitting. A model that is too simple has high error on both training and test data. Table 9 compares three linear models of increasing information: a constant prediction (the training mean), a linear model on the four continuous weather features only, and the full 23-feature linear model. The first two explain almost none of the variance (test R^2 of 0.000 and 0.027) and have about three times the error of the full model, which is the signature of underfitting: the weather variables alone cannot capture the strong daily and weekly pattern of traffic.

Table 9. Underfitting: simple models against the full linear model.

Model	Features	Train MSE	Test MSE	Test R^2
Constant (training mean)	1	3,966,481	3,871,753	0.000
Linear, 4 continuous features	4	3,864,625	3,766,107	0.027
Linear, all 23 features	23	1,182,480	1,173,326	0.697

Overfitting. On the full training set the sample size (38,563) is much larger than the number of parameters (23 to 88), so degrees 2 to 4 do not overfit; test MSE changes by less than 0.2% (Table 8). To show classical overfitting, polynomial models of degree 1 to 4 were trained on a random sample of only 300 training records, using the four continuous features, and evaluated on the full test set (Table 10). The training error falls steadily as the degree increases, while the test error is lowest for degree 1, rises slightly at degree 3 (6.9% above degree 1) and then jumps to 14 times the degree-1 value at degree 4 with 70 parameters for 300 observations. This is textbook high variance. The test period also comes after the training period, so part of the increase can also reflect extrapolation and a shift in the data.

Table 10. Overfitting in the controlled experiment with 300 training records.

Degree	Parameters	Train MSE	Test MSE
1	5	3,618,657	3,861,271
2	15	3,387,335	3,871,171
3	35	3,229,504	4,129,223
4	70	3,054,200	53,919,983

Degree 5 on the full data is a different case. Both the training MSE (3,930,451) and the test MSE (3,833,350) are about 3.3 times larger than for degree 4 and R^2 drops to about 0.01. Training error also collapses, so this is not variance overfitting. The design matrix has a condition number of 1.8×10^{27}, far above the reciprocal of double-precision machine epsilon (about 4.5×10^{15}), so the least-squares problem is numerically singular and the fit is unreliable. Degrees 3 and 4 also exceed this threshold (Table 8) yet fit well, so the condition number alone does not explain where the fit breaks down; the exact mechanism was not investigated.

3.5 Ridge and lasso regression

The regularisation strength α of ridge (L2) and lasso (L1) regression was chosen by 5-fold time-series cross-validation on the training set, scored by mean squared error. In time-series cross-validation each fold validates on a block that comes after its training block, so no future data leak into training. The ridge grid was 0.001 to 1000 and the lasso grid 0.0001 to 10. Ridge selects $\alpha = 100$ and lasso selects $\alpha = 10$ (Tables 11 and 12).

Table 11. Ridge cross-validation MSE.

α	CV MSE
0.001	1,197,299
0.01	1,197,299
0.1	1,197,297
1	1,197,279
10	1,197,124
100	1,196,747
1000	1,245,418

Table 12. Lasso cross-validation MSE.

α	CV MSE
0.0001	1,197,300
0.001	1,197,300
0.01	1,197,293
0.1	1,197,232
1	1,196,928
10	1,195,765

The cross-validation curve is almost flat for small α: ridge MSE changes by less than 0.05% between α = 0.001 and α = 100 and then rises sharply at α = 1000. The selected lasso value is the largest in its grid, so a larger α was not tried. With α = 10 the lasso sets 13 of the 23 coefficients to exactly zero: all 11 weather-condition indicators, the holiday flag and the sine of the month. The retained features are the hour-of-day terms, weekend flag, temperature, rainfall, snowfall, cloud cover, day-of-week terms and the cosine of the month. Because regularisation changes the test error by only 0.1% to 0.3%, the unregularised linear model is not overfitting here: with 38,563 records and 23 features there is little variance for regularisation to remove. For some of the smaller α values scikit-learn reported that the lasso coordinate descent had not fully converged within 5,000 iterations.

3.6 Comparison of all regression models

Table 13 compares all models with MSE, RMSE and R^2 on the training and test sets, as required.

Table 13. Regression results (MSE in vehicles² per hour², RMSE in vehicles per hour).

Model	Train MSE	Train RMSE	Train R^2	Test MSE	Test RMSE	Test R^2
Linear, gradient descent	1,183,099	1,087.7	0.7017	1,171,125	1,082.2	0.6975
Polynomial, degree 1 (= OLS)	1,182,480	1,087.4	0.7019	1,173,326	1,083.2	0.6969
Polynomial, degree 2	1,178,655	1,085.7	0.7028	1,166,996	1,080.3	0.6986
Polynomial, degree 3	1,178,157	1,085.4	0.7030	1,166,866	1,080.2	0.6986
Polynomial, degree 4	1,175,760	1,084.3	0.7036	1,164,935	1,079.3	0.6991
Polynomial, degree 5	3,930,451	1,982.5	0.0091	3,833,350	1,957.9	0.0099
Ridge (α = 100)	1,182,764	1,087.5	0.7018	1,172,175	1,082.7	0.6972
Lasso (α = 10)	1,187,423	1,089.7	0.7006	1,176,769	1,084.8	0.6961

All sensible linear and polynomial models reach a test R^2 between 0.696 and 0.699, an RMSE of about 1,080 vehicles per hour. The differences between them are tiny (about 1% of the MSE between the best and the worst), so the ranking should not be over-interpreted: degree 4 has the lowest test MSE, but degree 1, gradient descent, ridge and lasso are practically equivalent. The limit is set by information, not by the model: the features describe weather and calendar only, and the remaining error comes from effects such as accidents, roadworks and events that are not in the data. Figure 7 shows predicted against actual traffic for the gradient-descent model on the test set; the daily rush-hour pattern is captured but the spread around the diagonal is large.

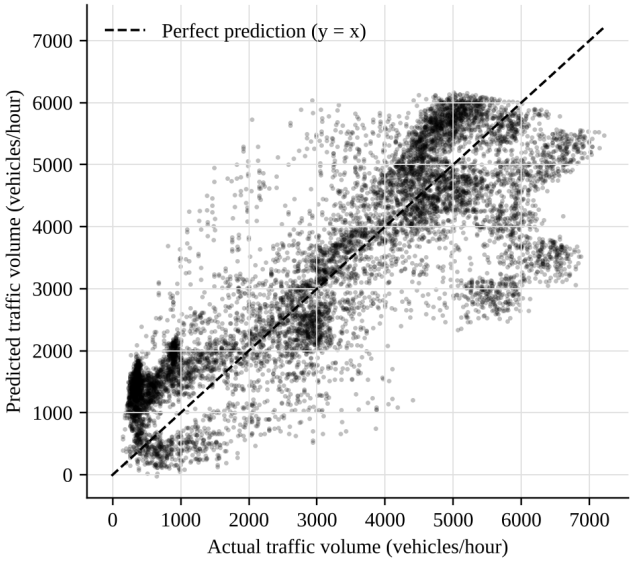


Figure 7. Predicted against actual traffic volume on the test set (gradient-descent linear regression).

3.7 Effect of the rainfall outlier

Without capping, the single 9,831.3 mm record gives rainfall a very large standardised range. The OLS test MSE then changes only from 1,173,326 (capped) to 1,172,720 (uncapped), a difference of 0.05%, but the fitted rainfall coefficient changes sign, from −34.9 to +3.7 vehicles per hour per standard deviation. A positive coefficient would wrongly say that rain increases traffic. Capping is therefore kept for data-quality and interpretability reasons, not because it improves test error.

3.8 Summary of Part 2

All required elements are covered: gradient-descent linear regression, polynomial regression, ridge and lasso, MSE, RMSE and R^2 for every model, cases of underfitting (simple models) and overfitting (degree 4 on 300 records), and cross-validated selection of the regularisation parameter.

4. Part 3A: K-Means Clustering from Scratch

4.1 Data

The gas-turbine data are hourly measurements from 2011 to 2015 (five yearly files, 36,733 rows). After removing 7 exact duplicate rows, 36,726 records remain. The 11 features are ambient temperature (AT), ambient pressure (AP), ambient humidity (AH), air filter difference pressure (AFDP), gas turbine exhaust pressure (GTEP), turbine inlet temperature (TIT), turbine after temperature (TAT), turbine energy yield (TEY), compressor discharge pressure (CDP), carbon monoxide (CO) and nitrogen oxides (NOx). Each feature is standardised to zero mean and unit variance, so that no variable dominates the Euclidean distance.

4.2 Algorithm

K-means is implemented with Lloyd's algorithm in NumPy. K initial centroids are chosen as random data points. The algorithm alternates between assigning each record to its nearest centroid and moving each centroid to the mean of its records, and stops when the largest centroid movement is below 10^{-4} or after 300 iterations. An empty cluster is re-seeded at the record farthest from its centroid. The algorithm finds a local minimum, so it is run from 10 random initialisations for each K and the run with the smallest WCSS is kept:

WCSS = sum, over all clusters k and all records x_i in cluster k, of $||x_i - \mu_k||^2$, where μ_k is the mean of cluster k.

4.3 Results for K = 2 to 6

Table 14 gives the WCSS for K = 2, 3, 4, 5 and 6. The K = 1 row is the total variance around the overall mean and is added only as a baseline for the first reduction. The silhouette score was computed on a fixed random sample of 8,000 records for speed.

Table 14. K-means WCSS and silhouette score for K = 1 to 6.

K	Best WCSS	Reduction from K − 1 (%)	Mean WCSS of 10 starts	Worst WCSS of 10 starts	Silhouette	Iterations
1 (baseline)	403,986	–	–	–	–	–
2	276,942	31.45	276,942	276,942	0.327	49
3	213,958	22.74	213,958	213,958	0.286	28
4	180,981	15.41	182,993	191,042	0.255	49
5	157,444	13.01	161,254	165,171	0.274	35
6	142,087	9.75	147,748	153,206	0.253	28

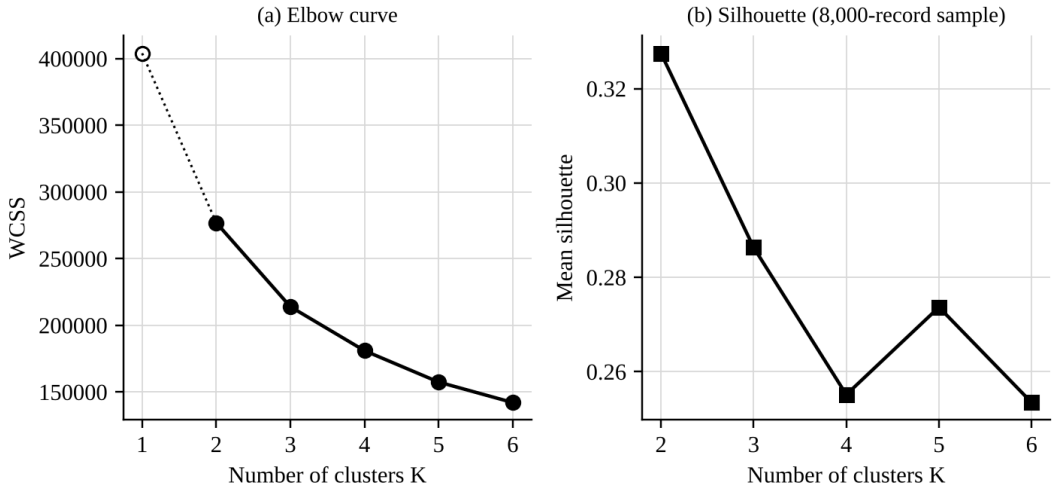


Figure 8. (a) Elbow curve of WCSS against K (the open circle at K = 1 is the baseline); (b) mean silhouette score.

WCSS decreases at every step, by 31.4% from K = 1 to 2, then 22.7%, 15.4%, 13.0% and 9.8%. The curve bends gradually and has no sharp elbow. The silhouette score is highest for K = 2 (0.327) and lower for K = 3 (0.286), so the two criteria do not point to a single best K. K = 3 is used below as a practical choice because the reduction slows noticeably after it and it separates the operating range into low, medium and high load (Table 15), not because it is the unique optimum. Table 14 also shows why restarts matter: for K = 2 and K = 3 all ten starts reach the same WCSS, while for K ≥ 4 the worst start is about 5% to 8% worse than the best.

Table 15. Profile of the K = 3 clusters (means in original units).

Cluster	Records	Mean TEY	Mean CDP	Mean TIT	Mean AT
0	8,435	112.2	10.62	1054.1	16.1
1	19,203	132.8	11.96	1084.8	18.4
2	9,088	154.7	13.61	1099.7	17.6

The clusters are ordered by turbine load: cluster 0 has the lowest mean energy yield, compressor pressure and turbine inlet temperature and cluster 2 the highest, while the mean ambient temperature is similar. The K = 3 partition therefore mainly separates low, medium and high load operation.

5. Part 3B: Principal Component Analysis from Scratch

5.1 Method

PCA is implemented with the covariance matrix and an eigenvalue decomposition in NumPy. The standardised data matrix X (36,726 × 11) is centred, the sample covariance matrix $C = X^{T}X / (N - 1)$ (11 × 11) is formed, and its eigenvalues λ_j and eigenvectors v_j are computed with a symmetric eigensolver and sorted in descending order of eigenvalue. The explained variance ratio is $EVR_j = \lambda_j / \Sigma\lambda$, and the data are reduced to k dimensions by $Z = X [v_1 \dots v_k]$. The eigenvectors are orthonormal to machine precision (largest deviation of $V^{T}V$ from the identity about 7×10^{-16}). The eigenvalues add up to 11.0003 rather than exactly 11, because the data were scaled with divisor N and the covariance uses N − 1; this does not affect the ratios.

5.2 Eigenvalues and explained variance

Table 16. Eigenvalues, explained variance ratio and cumulative explained variance.

Component	Eigenvalue	Explained variance (%)	Cumulative (%)
PC1	5.2801	48.00	48.00
PC2	2.1894	19.90	67.90
PC3	1.0390	9.45	77.35
PC4	0.8398	7.63	84.98
PC5	0.6887	6.26	91.24
PC6	0.4828	4.39	95.63
PC7	0.2597	2.36	97.99
PC8	0.1762	1.60	99.59
PC9	0.0343	0.31	99.91
PC10	0.0083	0.08	99.98
PC11	0.0021	0.02	100.00

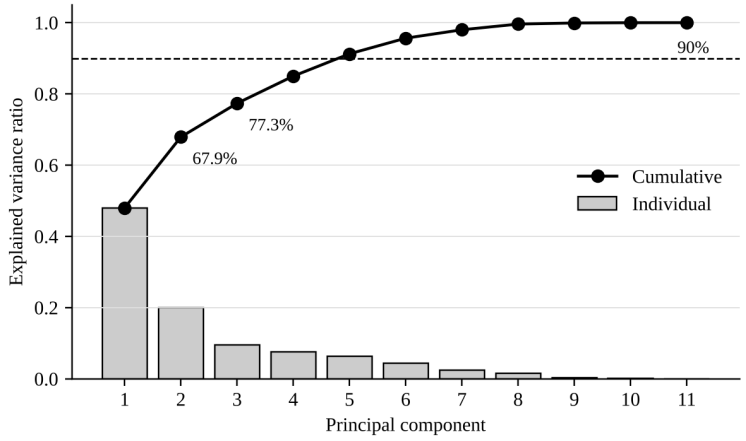


Figure 9. Individual and cumulative explained variance of the 11 principal components.

The first component explains 48.00% of the variance and the second 19.90%. Two components keep 67.90% and three keep 77.35%. Five components are needed to pass 90% (91.24%), and seven give 97.99%. The last four components together hold only 2.0% of the variance, which shows that the 11 sensors are strongly correlated.

5.3 Eigenvectors

Table 17 lists the eigenvectors (loadings) of the first three components. The sign of an eigenvector is arbitrary; only relative signs and sizes matter.

Table 17. Eigenvector loadings of the first three principal components.

Feature	PC1	PC2	PC3
AT	−0.051	0.605	−0.194
AP	−0.025	−0.379	−0.428
AH	0.108	−0.294	0.773
AFDP	−0.339	0.073	0.035
GTEP	−0.421	−0.058	−0.046
TIT	−0.405	0.072	0.101
TAT	0.279	0.344	0.181
TEY	−0.421	−0.131	0.045
CDP	−0.427	−0.084	−0.018
CO	0.281	−0.193	−0.352
NOX	0.110	−0.458	−0.097

PC1 is dominated by CDP, GTEP, TEY and TIT (loadings from −0.405 to −0.427) and AFDP (−0.339), with opposite-signed weight on TAT and CO (about +0.28). These are the variables that rise and fall together with turbine load, so PC1 can be read as a load axis. In this solution the sign is such that a larger PC1 means lower load, which agrees with Figure 10, where energy yield decreases from left to right. PC2 has its largest weights on ambient temperature (+0.605), NOx (−0.458), ambient pressure (−0.379), TAT (+0.344) and humidity (−0.294), and so mixes ambient conditions with emissions. PC3 is dominated by humidity (+0.773) with ambient pressure (−0.428) and CO (−0.352). These readings are interpretations of the loadings, not tested results.

5.4 Dimensionality reduction and visualisation

Projecting onto the first two and the first three components gives matrices of size 36,726 × 2 and 36,726 × 3 that keep 67.90% and 77.35% of the variance. Figure 10 shows the two-component projection, shaded by turbine energy yield, and Figure 11 shows the same projection with the K = 3 clusters.

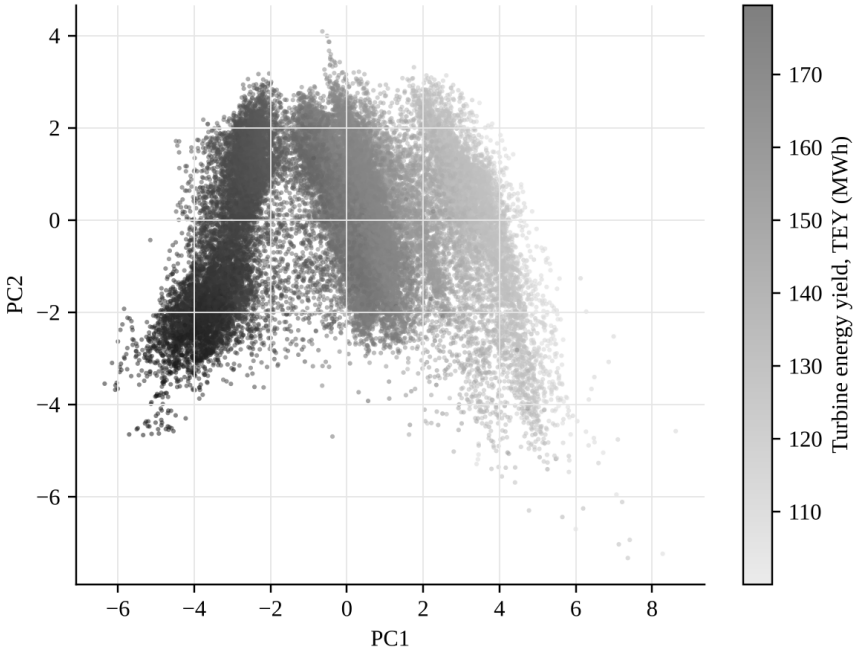


Figure 10. Data projected onto the first two principal components, shaded by turbine energy yield.

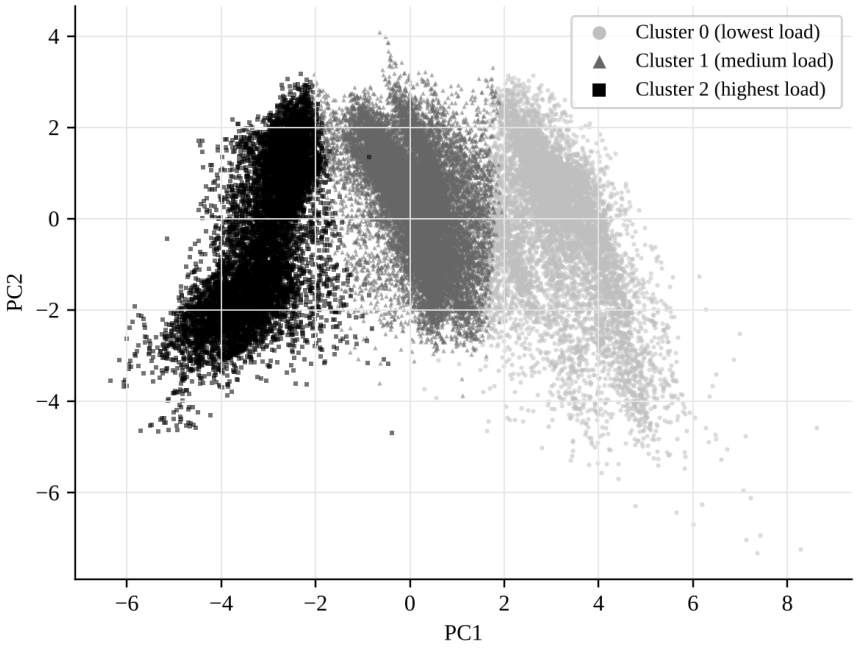


Figure 11. The same projection with the K-means (K = 3) cluster assignments.

The projection shows a smooth gradient of energy yield along PC1 and a curved, banana-shaped cloud rather than separate groups, which is consistent with the weak elbow in Section 4. The K = 3 clusters are separated mostly by vertical boundaries in PC1, i.e. by load level, which matches the cluster profiles in Table 15.

6. Limitations

The following points should be kept in mind when reading the results.

- Hyperparameters of the SVM were tuned on a 10,000-title subset with 3-fold cross-validation, and C = 10 is at the edge of the grid. KNN (k = 15) and the decision tree (unlimited depth) also chose values at the edge of their grids; larger values were not tested.

- The TF-IDF vocabulary was fitted once on the whole training set before cross-validation instead of being refitted inside each fold. This can make the cross-validation scores slightly optimistic. The test set was never used for fitting.
- All results come from a single train/test split and one random seed. Re-running the classification experiments in a different software environment changed KNN and decision-tree accuracy by about 0.1 percentage point (the SVM result was unchanged), without changing any conclusion.
- Gradient descent stopped at the iteration limit, and for some small α values the lasso solver reported incomplete convergence.
- The regression test period lies after the training period, so error figures include the effect of any change in traffic patterns over time.
- The silhouette scores use a sample of 8,000 records. The choice K = 3 is a judgement; neither the elbow nor the silhouette identifies it uniquely.
- The optional CNN bonus (Part 1d) was not attempted.

7. Reproducibility and Files

The code submission contains the data, scripts, saved metrics and figures. From the project root the experiments are run with:

```
python experiments/run_clustering.py
python experiments/run_regression.py
python experiments/run_classification.py
python experiments/run_diagnostics.py
```

The first three scripts write the main tables and figures to outputs/*/metrics and outputs/*/plots. The script run_diagnostics.py writes the supporting tables used in this report (cross-validation tables, the decision-tree depth sweep, the 300-record experiment, condition numbers, the gradient-descent check, the rainfall test, the silhouette scores) to outputs/diagnostics. The source code is in src/ (classification, regression, clustering and common utilities) and all settings are in config.yaml.

8. References

UCI Machine Learning Repository datasets: PriceRunner Product Classification (ID 837); Metro Interstate Traffic Volume (ID 492); Gas Turbine CO and NOx Emission Data Set (ID 551).